

ПРЕРЕКВИЗИТЫ

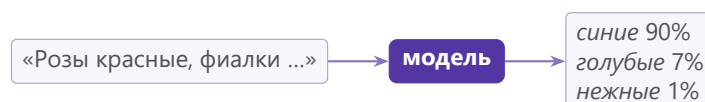
Как устроены языковые модели

Ликбез без формул: как текст превращается в числа, что происходит внутри трансформера, откуда модель умеет отвечать и чего принципиально не может. Читается за полчаса. Ровно столько, чтобы термины первых лекций были не холодными, а инженерные решения курса (память, кэш, RAG) выглядели не магией, а следствием устройства.

Одна вещь, которую делает модель

Большая языковая модель (LLM) в своей основе делает *одно*: смотрит на текст и предсказывает, каким будет **следующий токен**. Токен это кусок текста: слово, часть слова или знак.

Она не выдаёт один вариант, а строит **распределение вероятностей** по всему словарю: «дальше *синие* с вероятностью 90%, *голубые* 7%, *нежные* 1%». Из этого распределения выбирается один токен, дописывается к тексту, и всё повторяется. Так, по одному токenu, рождается весь ответ.



Вот и весь «интеллект» на нижнем уровне. Гладкие ответы, рассуждения, код: всё это следствие того, что модель научилась предсказывать следующий токен *очень* хорошо, прочитав огромный корпус текста. Дальше разберём, как именно она это делает, по шагам: от букв к числам, от чисел к вероятностям, от вероятностей обратно к буквам.

СВЯЗЬ С КУРСОМ

На первой лекции это превратится в **температуру**: ручку, которая говорит, брать ли всегда самый вероятный токен (temperature 0, предсказуемо, то, что нужно агенту) или иногда выбирать менее вероятные (выше значит разнообразнее). Пощупать распределение можно уже сегодня в ноутбуке `tokens_playground.ipynb`.

Шаг первый: из текста в числа

Нейросеть работает с числами, а не с буквами. Поэтому текст сначала надо превратить в числа, и делается это в два приёма.

Токенизация: режем текст на куски

Модель не работает ни с буквами (слишком мелко, длинные последовательности), ни с целыми словами (слишком много разных слов, и что делать с незнакомыми). Она работает с промежуточной единицей, **токеном**. Словарь токенов строится автоматически перед обучением: алгоритм (обычно **BPE**, byte pair encoding) смотрит на огромный корпус и склеивает самые частые сочетания символов в один токен. Частое слово `the` становится одним токеном, редкое токенизация распадается на несколько кусков.

- Словарь фиксирован и конечен: обычно 50–200 тысяч токенов. Любой текст на любом языке раскладывается на них, потому что на самом дне лежат отдельные байты.
- Словарь строился в основном на английском тексте, поэтому английские слова часто целые токены, а кириллица режется мельче. Русский текст того же смысла занимает больше токенов.
- Есть **спецтокены**: начало и конец текста, границы реплик. Роли `system`, `user`, `assistant` из первой лекции на этом уровне тоже просто особые токены-разделители.

Эмбединг: у каждого токена свой вектор

Каждому токenu словаря сопоставлен **вектор** из нескольких тысяч чисел, и эта таблица тоже обучается. Токены с похожим употреблением получают похожие векторы. Так текст превращается в последовательность векторов: с этого момента внутри модели живут только числа.

К вектору каждого токена подмешивается информация о его **позиции** в тексте. Без неё модель видела бы «мешок» токенов и не отличала «кошка съела мышь» от «мышь съела кошку».

СВЯЗЬ С КУРСОМ

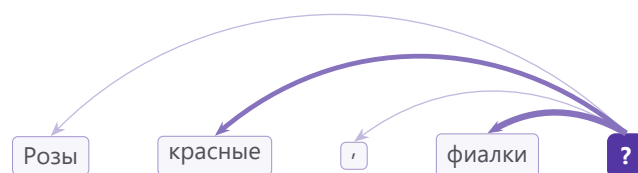
Отсюда сразу два следствия для курса. Первое: **за токены платят**, вход и выход по-разному, и русский обходится дороже английского, потому что режется мельче (первая лекция). Второе: на второй неделе появятся **эмбединги текста** для поиска по смыслу. Это родственная идея, но другая модель: там один вектор описывает целый абзац, а здесь вектор у каждого токена.

Шаг второй: внутри трансформера

Внутри модели стоит архитектура **трансформер** (Google, 2017). Это стопка одинаковых **слоёв**, от нескольких десятков до сотни с лишним, и векторы токенов проходят через неё снизу вверх. Каждый слой делает две вещи.

Внимание: токены смотрят друг на друга

В блоке **внимания** (attention) вектор каждого токена «оглядывается» на все токены до него и решает, какие из них важны прямо сейчас. Для слова «синие» в стихе про фиалки внимание вытянет «фиалки» и «красные», а не запятую. Технически это взвешенная сумма чужих векторов, где веса модель подбирает сама. Голов внимания в слое несколько, каждая ищет своё: одна согласование, другая ссылки на упомянутое ранее, третья структуру фразы.



толщина стрелки это вес внимания: на что модель смотрит, предсказывая следующий токен

Важно: каждый токен смотрит только *назад*, на то, что уже есть. Вперёд смотреть не на что, будущего текста ещё нет. Это и делает модель генератором, а не просто читателем.

Полносвязный блок: где лежит знание

После внимания вектор каждого токена проходит через **полносвязный блок** (feed-forward). Если внимание отвечает за то, *откуда* взять информацию, то этот блок за то, *что с ней сделать*: применить закономерности, извлечь факт, преобразовать. Исследования показывают, что именно здесь хранится большая часть фактического знания модели, вроде «Париж столица Франции».

Слой за слоем: вектор обрastaет смыслом

На входе вектор токена «синие» знает только про само слово. После первых слоёв он уже «понимает», что стоит в стихотворении, в роли цвета, после «фиалки». К верхним слоям вектор последнего токена вобрал в себя всё, что нужно, чтобы предсказать, что будет дальше. Модель не хранит промежуточных выводов явно, но каждый слой уточняет представление.

Выход: от вектора к вероятностям

Вектор последнего токена с верхнего слоя умножается на таблицу словаря, и для каждого из десятков тысяч возможных токенов получается число, **логит**: насколько он подходит. Функция **softmax** превращает эти числа в вероятности, которые в сумме дают единицу. Это и есть то распределение, с которого мы начали.



Параметры: что такое «веса»

Все таблицы, о которых шла речь (эмбеддинги, веса внимания, полносвязные блоки, выходная таблица), это **параметры** модели, они же веса. У современных моделей их от единиц до сотен миллиардов. Обучение это подбор этих чисел так, чтобы предсказание следующего токена на корпусе становилось всё точнее. Знание модели живёт не в базе данных, а размазано по этим миллиардам чисел, и вытащить его можно только одним способом: спросить.

Почему это не «понимание» в человеческом смысле

Модель не хранит факты как записи, не имеет намерений и не проверяет себя. Она очень хорошо угадывает, какой текст был бы уместен дальше. Часто уместный текст совпадает с правдой, потому что в корпусе правды больше. Но там, где данных мало, уместное и правдивое расходятся, и модель выбирает уместное. Отсюда галлюцинации, о них ниже.

Шаг третий: генерация по одному токenu

Ответ рождается циклом. Прогнали текст через модель, получили распределение, выбрали токен, дописали его к тексту, прогнали снова. И так до спецтокена «конец ответа» или до лимита. Такой способ называется **авторегрессионным**: каждый новый токен зависит от всех предыдущих, включая только что сгенерированные.

Выбор из распределения: сэмплирование

Как именно выбирать токен из распределения, решаете вы, а не модель.

- **Жадный выбор**: всегда самый вероятный. Предсказуемо, но может заикливаться на повторах.
- **Температура**: делит логиты перед softmax. Меньше единицы обостряет распределение (лидер побеждает почти всегда), больше единицы сглаживает (шансы выравниваются). Ноль эквивалентен жадному выбору.
- **Тор-р**: отбросить хвост маловероятных токенов и выбирать только среди тех, чья суммарная вероятность набирает p . Защищает от совсем нелепых вариантов при высокой температуре.

Почему выход дороже входа: KV-кэш

Когда модель генерирует сотый токен, ей нужно внимание ко всем предыдущим 99. Пересчитывать их представления заново на каждом шаге расточительно, поэтому промежуточные результаты внимания для уже обработанных токенов сохраняются в **KV-кэше**. Входной текст (промпт) обрабатывается за один проход целиком и параллельно, это быстро. А каждый выходной токен требует отдельного прогона через все слои, один за другим, и это медленно. Отсюда две вещи, которые вы увидите в счетах.

СВЯЗЬ С КУРСОМ

Первое: **выходные токены стоят в 3–5 раз дороже входных** у всех провайдеров, и это не жадность, а физика: выход генерируется последовательно, вход обрабатывается разом. Второе: **prompt caching** с первой лекции это ровно сохранение KV-кэша для повторяющегося начала запроса. Провайдер не пересчитывает системный промпт и документы, если они не изменились, и берёт за них в разы меньше. Поэтому стабильное кладут в начало, а меняющееся в конец: кэш работает только для неизменного префикса.

Ещё одно следствие: раз токены рождаются по одному, их можно отдавать сразу, не дожидаясь конца. Это **стриминг**: текст в чате появляется по словам не для красоты, а потому что так устроена генерация.

Почему контекст ограничен и почему длинный контекст ненадёжен

Контекстное окно это максимум токенов, которые модель может держать в одном прогоне: промпт плюс ответ. Ограничение не произвольное.

- Внимание сравнивает каждый токен с каждым. Удвоили длину текста, работы стало вчетверо больше. Память под KV-кэш растёт вместе с длиной. Поэтому у окна есть техническая граница, а длинные запросы дороже и медленнее.
- Позиционная информация обучена на текстах ограниченной длины. За пределами того, что модель видела при обучении, она ориентируется хуже.
- Даже внутри окна внимание размывается: чем больше токенов, тем труднее выделить нужный. Модели заметно лучше находят факты в начале и в конце длинного контекста, чем в середине. Этот эффект называют «потерян в середине».

СВЯЗЬ С КУРСОМ

Отсюда вся четвёртая лекция про память: контекстное окно это **бюджет**, а не склад. В него кладут минимум, которого хватит для следующего шага, а остальное держат снаружи и достают по необходимости. На лекции покажем эффект «потерян в середине» на живом эксперименте.

Откуда модель умеет отвечать: стадии обучения

Модель не «запрограммировали» отвечать. Её обучили, и это несколько разных стадий с разными целями.

1. **Предобучение.** Модель читает корпус в триллионы токенов: интернет, книги, код, статьи. Единственная задача: предсказать следующий токен, единственный сигнал: насколько промахнулась. Это самая дорогая стадия, месяцы на тысячах ускорителей. На выходе модель знает язык, массу фактов и умеет продолжать любой текст. Но ведёт себя как автопродолжатель: на вопрос может выдать ещё десять похожих вопросов, потому что в интернете за вопросом часто идёт другой вопрос.
2. **Инструктивное дообучение (SFT).** Модели показывают десятки тысяч пар «инструкция и хороший ответ», написанных людьми. Она учится формату разговора: на вопрос отвечают, а не продолжают. Здесь же появляется привычка слушаться системного промпта.
3. **Обучение на предпочтениях (RLHF и его родственники).** Людям показывают два ответа модели на один запрос, они выбирают лучший. На этих выборах учат отдельную модель-судью или напрямую подстраивают основную (метод DPO). Модель сдвигается к тому, что люди считают полезным, вежливым и безопасным. Побочный эффект: излишняя услужливость, склонность соглашаться и извиняться.

4. Обучение рассуждать. Новое поколение моделей дополнительно учат думать перед ответом: генерировать цепочку промежуточных шагов, проверять себя, и только потом отвечать. Эти «мысли» тоже токены, за них тоже платят, даже если провайдер их не показывает. Такие модели сильнее в математике, коде и многшаговых задачах, но медленнее и дороже.

Поэтому современная модель слушается инструкций, отвечает по делу и рассуждает: это результат стадий 2–4, а не магия и не «понимание».

Семейства моделей и что между ними общего

- **Размер.** Считают в миллиардах параметров. Маленькие модели (единицы миллиардов) быстрые и дешёвые, но слабее в рассуждениях. Большие умнее и дороже. Провайдеры часто выпускают линейку: младшая для рутины, старшая для сложного. На курсе будем именно так и делать: рутину на дешёвой, проверку на сильной.
- **Открытые веса против API.** Часть моделей выложена целиком, их можно запускать у себя (Llama, Qwen, Mistral, DeepSeek). Другие доступны только через API провайдера (Claude, GPT, Gemini). Устроены все одинаково, разница в доступе, цене и приватности.
- **Дистилляция.** Маленькую модель учат повторять ответы большой. Так появляются быстрые модели, которые «унаследовали» часть умений старших.
- **Модели-эмбеддеры.** Отдельный класс: не генерируют текст, а выдают один вектор на весь текст, чтобы сравнивать смыслы. Тот же трансформер, но обученный на другую задачу. Понадобятся на второй неделе для поиска и RAG.

Чего модель НЕ умеет, и почему существует этот курс

Всё выше складывается в четыре принципиальных ограничения. Каждое следует из самого устройства, а не из чьей-то недоработки.

- **Нет памяти.** Между двумя вызовами модель не помнит ничего. Она чистая функция: что прислали на вход, из того и родился ответ. Помнит не модель, а ваш код, который заново шлёт ей всю историю.
- **Отсечка знаний.** Знание живёт в весах, а веса зафиксированы в момент окончания обучения. Про вчерашние новости и ваши внутренние документы модель не знает ничего и узнать сама не может.
- **Галлюцинации.** Модель предсказывает *правдоподобное* продолжение, а не *правдивое*. Если фактов нет, она уверенно придумает, и это будет звучать убедительно, потому что убедительность и есть то, чему её учили.
- **Недетерминированность.** На один и тот же вопрос может ответить по-разному: она выбирает из распределения. Даже при температуре ноль ответы иногда чуть плавают из-за особенностей вычислений на ускорителях.

СВЯЗЬ С КУРСОМ

Вот здесь и начинается весь курс. **Инструменты** и **tool calling** дают модели дотянуться до мира: посчитать, поискать, сходить в базу. **RAG** и **память** чинят отсутствие памяти и отсечку знаний, подкладывая нужные документы в контекст. **Evals** ловят галлюцинации. Агент это леса, которые достраивают вокруг модели то, чего ей самой не хватает по устройству.

Пять мифов, которые мешают работать

МИФ

«**Модель ищет ответ в интернете**». Нет. Без инструментов у неё есть только веса и то, что вы прислали. Если кажется, что она знает свежее, значит, провайдер сам подключил ей поиск как инструмент. Это и есть tool calling.

МИФ

«**Модель меня помнит**». Нет. Каждый вызов начинается с чистого листа. Если чат «помнит» прошлые сообщения, значит, приложение шлёт их заново каждый раз. И платит за это.

МИФ

«**Модель может проверить факт**». Нет. Она может только сгенерировать текст, похожий на проверку. Настоящая проверка это вызов инструмента и сравнение с внешним источником. Или второй, независимый прогон, о чём будет лекция про критика.

МИФ

«**Больше параметров, значит всегда лучше**». Не всегда. Большая модель дороже и медленнее, а для классификации или извлечения полей младшая справится не хуже. Выбор модели это замер на своих задачах, а не размер.

МИФ

«**Если хорошо попросить, она перестанет ошибаться**». Промпт двигает поведение, но не чинит устройство. Инструкция «не выдумывай» снижает, но не убирает галлюцинации. Убирают их инструменты, документы в контексте и проверка ответов.

Глоссарий

Чтобы на лекциях ни одно слово не было холодным. Одна строка на термин, сгруппированы по темам.

УСТРОЙСТВО

Токен Кусок текста, которым модель меряет вход и выход: слово, часть слова или знак.

ВРЕ Алгоритм, который строит словарь токенов, склеивая самые частые сочетания символов.

Эмбеддинг токена Вектор чисел, сопоставленный каждому токenu словаря. Похожие токены, похожие векторы.

Трансформер Архитектура модели: стопка слоёв, в каждом внимание и полносвязный блок.

Внимание (attention) Механизм, которым токен выбирает, какие предыдущие токены важны для предсказания.

Параметры (веса) Миллиарды чисел, в которых живёт всё знание модели. Подбираются при обучении.

Логиты и softmax Числа-оценки для каждого токена словаря и их превращение в вероятности.

Распределение вероятностей Что модель выдаёт на каждом шаге: шансы всех вариантов следующего токена.

ГЕНЕРАЦИЯ И ЦЕНА

Авторегрессия Генерация по одному токenu: каждый следующий зависит от всех предыдущих.

Температура Ручка случайности выбора следующего токена. 0 предсказуемо, выше разнообразнее.

Топ-р Отбросить маловероятный хвост и выбирать только среди верхних вариантов.

KV-кэш Сохранённые промежуточные результаты внимания, чтобы не пересчитывать контекст на каждом шаге.

Prompt caching Повторное использование KV-кэша для неизменного начала запроса. Дешевле в разы.

Контекстное окно Максимум токенов, которые модель видит за раз. Всё сверх не влезает.

Стриминг Отдача токенов по мере генерации, а не после конца ответа.

usage Поле ответа с числом токенов на входе и выходе и стоимостью. Читать всегда.

ОБУЧЕНИЕ И ВИДЫ МОДЕЛЕЙ

Предобучение Чтение огромного корпуса с единственной задачей предсказать следующий токен.

SFT Инструктивное дообучение на парах «инструкция, хороший ответ». Учит формату разговора.

RLHF, DPO Обучение на человеческих предпочтениях между ответами. Учит быть полезной и безопасной.

Reasoning-модель Модель, обученная генерировать цепочку рассуждений перед ответом. Точнее, но дороже.

Отсечка знаний Дата, на которой закончилось обучение: свежее модель не знает.

Открытые веса Модель, которую можно скачать и запустить у себя, в отличие от доступной только по API.

Дистилляция Обучение маленькой модели повторять ответы большой.

Модель-эмбеддер Отдельный класс: не генерирует текст, а выдаёт один вектор на весь текст для поиска по смыслу.

РАБОТА С МОДЕЛЬЮ

Промпт Текст, который вы отправляете модели. Системный, пользовательский, а в истории и ответы модели.

Роли system / user / assistant Разметка сообщений: правила, вопрос пользователя, прошлый ответ модели.

Few-shot Показать модели 2–5 примеров «вход, выход» вместо словесного описания формата.

Structured output Ответ строго по схеме (JSON), чтобы его читала программа, а не человек.

Tool calling Механизм, которым модель просит вызвать вашу функцию с аргументами.

Галлюцинация Уверенно звучащий, но выдуманный ответ, когда у модели нет фактов.

Провайдер Компания, отдающая модель по API: Anthropic, OpenAI, Google и другие.

OpenRouter Единая точка доступа ко всем моделям через один ключ и один формат запроса.

И последнее на сегодня: короткая анкета (файл `anketa.md`). Три минуты, они помогут нам настроить курс под вас. Увидимся на первой лекции.